

Milestone 3 Documentation: Advanced Frontend Architecture and UI/UX Implementation

Project Title: Developing an AI System for Personalized Content
Creation in Media

Intern Name: Naina Anil Pandey

Internship Program: Infosys Springboard 6.0

Milestone 1 Period: March 02, 2026 – March 15, 2026

1. Introduction

After successfully establishing a robust Large Language Model (LLM) backend with fallback mechanisms in **Milestone 2**, the focus of **Milestone 3** shifted toward frontend engineering and User Experience (UX) design.

The main objective of this milestone was to transform the application from a **basic prototype interface into a fully interactive Software-as-a-Service (SaaS) platform**.

To achieve this, the frontend architecture was redesigned using modern web technologies. The system was migrated from a **Streamlit-based interface to a decoupled React frontend**, allowing greater flexibility, scalability, and improved user experience.

This milestone emphasized:

- Building a **modern React-based frontend**
- Designing a **premium glassmorphic UI**
- Implementing **interactive animations**
- Establishing **secure client-server communication**
- Improving **state management and user session handling**

The result is a highly responsive interface that effectively integrates with the AI-powered backend system.

2. Architectural Evolution

To support a dynamic and scalable frontend, the system architecture was redesigned using a **decoupled client-server model**.

2.1. Decoupled Architecture

- The frontend was separated from the Python backend. This separation allows independent development, easier scalability, and improved state management.

2.2 Vite + React Integration

- The frontend was rebuilt using **React.js powered by Vite**. Vite significantly improves the development experience by providing:
 - Faster Hot Module Replacement (HMR)
 - Faster build performance
 - Efficient dependency handling

2.3 Cross-Origin Resource Sharing (CORS)

- Since the frontend and backend run on different ports, **CORS middleware** was added to the Flask backend.
- This enables secure communication between:
 - React Development Server → **Port 5173**
 - Flask Backend API → **Port 8000**
- This configuration allows seamless data exchange between the user interface and the AI backend engine.

3. Frontend Technology

The new user interface was developed using a modern and scalable technology stack.

3.1 React.js

React was used to build reusable UI components and manage dynamic user interactions through component-based architecture.

3.2 Tailwind CSS (v4)

Tailwind CSS was used for **utility-first styling**, enabling rapid development of complex layouts such as:

- Glassmorphic panels
- Animated mesh backgrounds
- Responsive design components

3.3 Framer Motion

Framer Motion was integrated to implement advanced animations including:

- Smooth page transitions
- Hover interactions
- Modal animations
- Staggered UI effects

3.4 React Router

React Router was used to implement **Single Page Application (SPA)** navigation. This allows users to navigate between pages without full page reloads, improving performance and user experience.

3.5 Next-Themes & Dark Mode

Dark and light themes were implemented using **Next-Themes and Tailwind dark mode configuration**.

User theme preferences are stored in **localStorage**, ensuring that the preferred theme persists across sessions.

3.6 Lucide React Icons

Lucide React was used for consistent and scalable iconography throughout the application interface.

4. Core UI Components Developed

The frontend interface was divided into multiple interactive pages, each designed to enhance usability and functionality.

4.1 Interactive Landing Page

The landing page acts as the first interaction point for users.

Key features include:

- Animated "**Dream. Manifest.**" hero section
- Glassmorphic UI design
- Smooth motion transitions

- A major feature of this page is the **Live Terminal Demo**, which simulates the behavior of the backend Cascade Engine.
- This was implemented using a **React useEffect-based typewriter animation**, which dynamically displays AI generation commands to demonstrate system functionality.

4.2 Authentication Flow (Auth.tsx)

- A secure authentication interface was developed with:
- Login and registration modes
- Animated UI transitions
- Glass-style authentication panels
- User authentication sessions are simulated by storing authentication tokens in **browser localStorage**, allowing session persistence across application reloads.

4.3 Intelligent Dashboard(app.jsx)

The Content Creator page functions as a parameter configuration panel where users define content generation attributes. The dashboard acts as the main control center for content generation, where users can visually select content types such as:

- LinkedIn Posts
- Twitter Threads
- Marketing Content
- Blog Snippets

Additionally, the dashboard includes Trending Quick Tools, which automatically populate generation parameters based on trending industry topics and hashtags.

Users can customize parameters including:

- Tone of content
- Target persona
- Content format
- Creativity level (scaled between 0.0 – 1.0)

Once parameters are configured, the payload is sent to the backend AI engine for content generation.

5. State Management and API Integration

To ensure reliable communication between the frontend and backend, robust state management and asynchronous API handling were implemented.

5.1 Session Persistence

The **App.jsx router** checks browser localStorage for active authentication tokens.

If no valid session exists, access to protected routes such as **/app** is restricted.

5.2 Asynchronous Data Fetching

Mock functions were replaced with real API calls using the **Fetch API**.

The frontend processes metadata returned by the backend, including:

- **engine_used**
- **validation_status**

These responses are displayed to the user through real-time notifications using the **Sonner toast notification library**.

5.3 Loading States

To prevent duplicate requests during AI processing, loading indicators were implemented.

These include:

- Animated spinners
- Processing messages
- Disabled submission buttons

This ensures a smooth and predictable user experience while the Gemini–Groq cascade engine processes requests.

6. Conclusion

Milestone 3 successfully transformed the backend AI infrastructure into a **fully interactive, user-friendly SaaS platform**.

By implementing a modern **React + Tailwind frontend architecture**, the system now provides:

- A visually appealing interface
- Smooth interactive animations
- Robust state management
- Seamless API communication with the backend AI engine

The result is a **premium AI-powered content generation platform** that abstracts the complexity of the multi-model backend and delivers a polished, professional user experience.

This milestone lays the foundation for **final deployment, scalability improvements, and advanced feature integration in the next stage of development**.